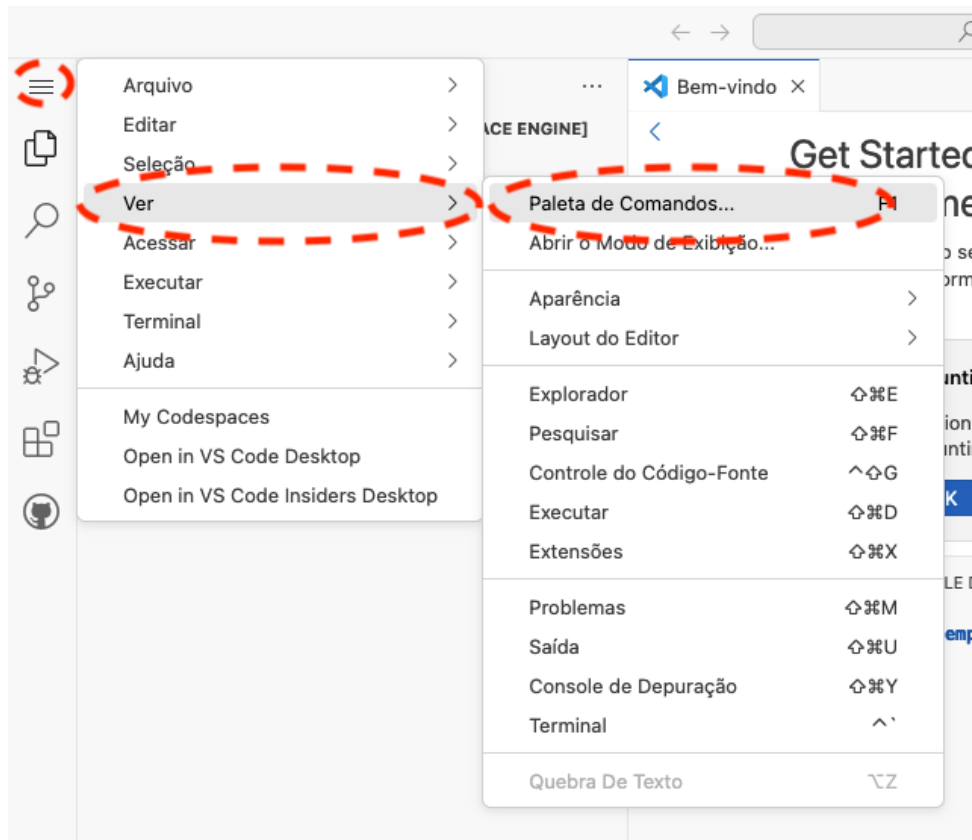


EXERCÍCIO DE TESTE UNITÁRIO COM JUNIT

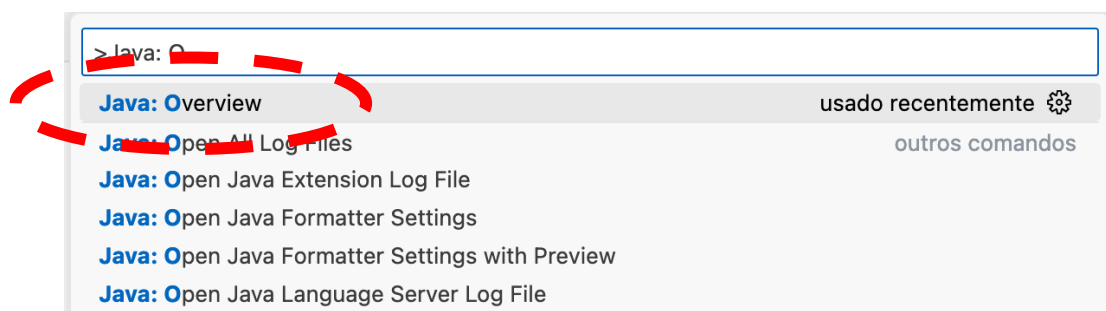
Configurando e usando o JUnit

Este exercício aborda a configuração e uso do JUnit para testes unitários.
Siga e execute os passos do roteiro.

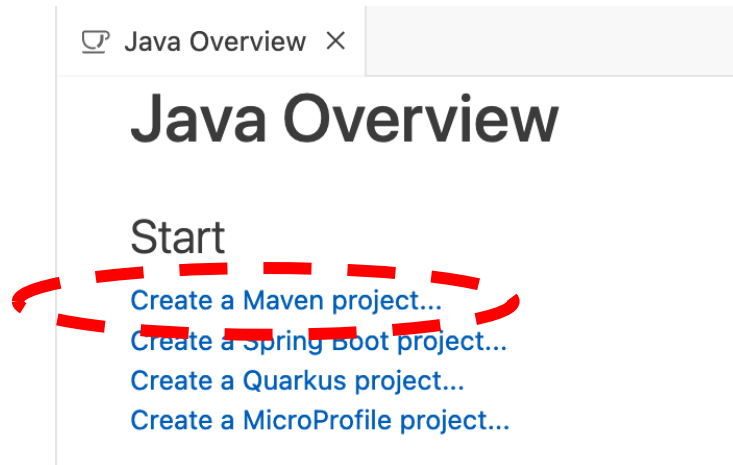
1. Crie um repositório (privado, Java) no GitHub para este exercício.
2. Inicie um Codespace a partir deste repositório.
3. No Codespace adicione o plugin 'Extension Pack for Java'
 - a. Por default, o 'Maven for Java' é instalado. Entretanto, se não for instalado, procure este plugin e o instale.
4. Crie um projeto Maven:
 - a. Inicie a paleta de comandos:



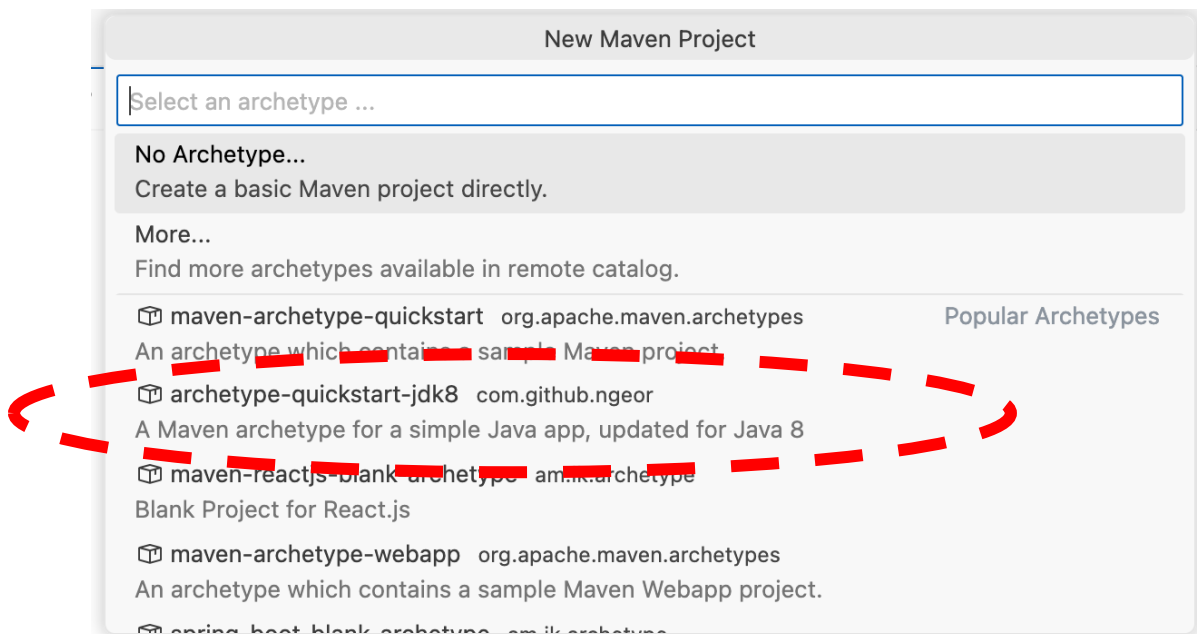
- b. Abra a visão geral do Java:



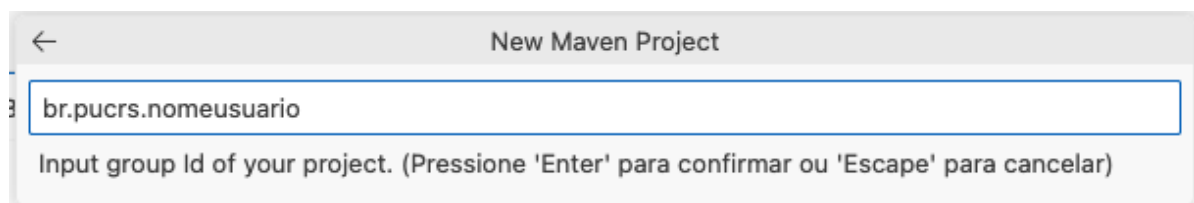
- c. Execute o comando de criar um novo projeto Maven:



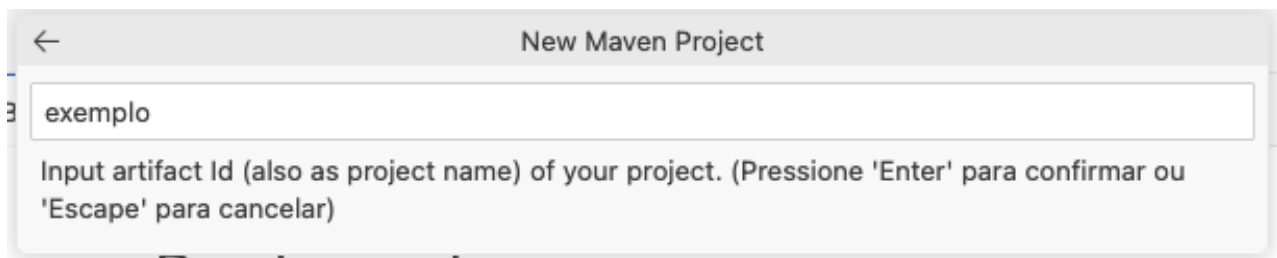
- d. Selecione um arquétipo (modelo). Para este roteiro use 'maven-quickstart-jdk8':



- e. Selecione a versão mais recente.
f. Nomeie o identificador do grupo com um domínio. Use o seu usuário de e-mail da PUCRS invertido.



- g. Nomeie o projeto:



- h. Confirme a pasta do projeto e aguarde o Maven realizar as configurações iniciais.
- i. No Terminal, indique a versão desejada para o projeto ou apenas confirme com ENTER.

```
[INFO] Using property: groupId = br.pucrs.nomeusuario  
[INFO] Using property: artifactId = exemplo  
Define value for property 'version' 1.0-SNAPSHOT: : 
```

- j. No Terminal, revise as configurações e confirme com ENTER:

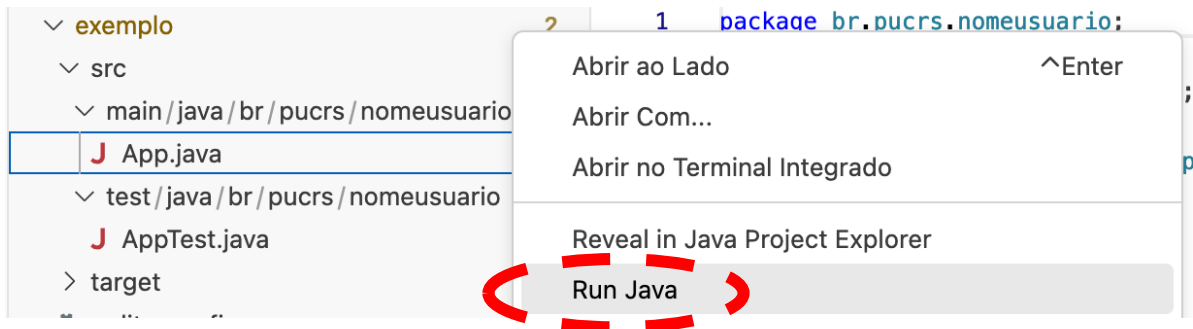
```
[INFO] Using property: package = br.pucrs.nomeusuario  
Confirm properties configuration:  
groupId: br.pucrs.nomeusuario  
artifactId: exemplo  
version: 1.0-SNAPSHOT  
package: br.pucrs.nomeusuario  
Y: : 
```

- k. O Maven fará configurações finais e apresentará uma mensagem confirmação que o projeto foi criado.
- l. Após a criação do projeto Maven explore a sua estrutura:

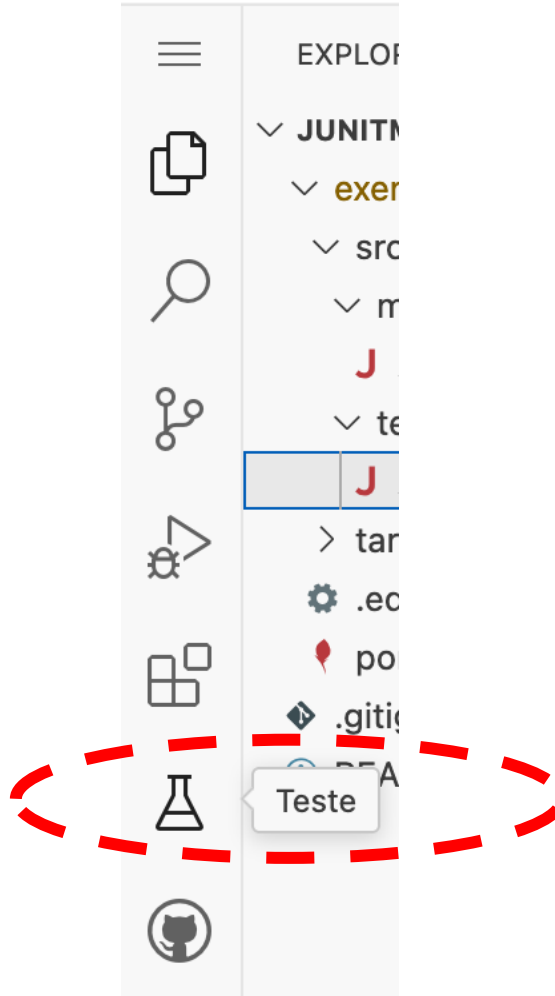


- Note que dentro da pasta src (códigos-fonte) há 2 pastas:
 - main ...: para implementar os códigos-fonte do sistema.
 - test ...: para implementar os códigos-fonte dos testes do sistema.

- l. Veja que na pasta teste há a classe AppTest.java que é gerada automaticamente.
- m. Compile o projeto pelo ciclo de vida Maven ou execute a classe App.java:



n. Nos atalhos à esquerda deve ter aparecido um ícone para testes:



o. Crie uma classe Java, chamada Conta, que será testada. Na subpasta de código da pasta main, crie a classe Conta.java (adapte o package para o seu projeto):

```

package br.pucrs.nomeusuario;

public class Conta {
    private double saldo;

    public Conta(double saldo) {
        this.saldo = saldo;
    }

    public double getSaldo() {
        return this.saldo;
    }

    public void setSaldo(double saldo) {
        this.saldo = saldo;
    }

    public void depositar(double valor) {
        saldo = saldo + valor;
    }

    public void sacar(double valor) {
        saldo = saldo - valor;
    }
}

```

- p. Crie uma classe de testes, chama ContaTest. Na subpasta de códigos de teste da pasta test, crie a classe ContaTest.java (adapte o package para o seu projeto):

```

package br.pucrs.nomeusuario;

import static org.junit.jupiter.api.Assertions.*;
import org.junit.jupiter.api.Test;

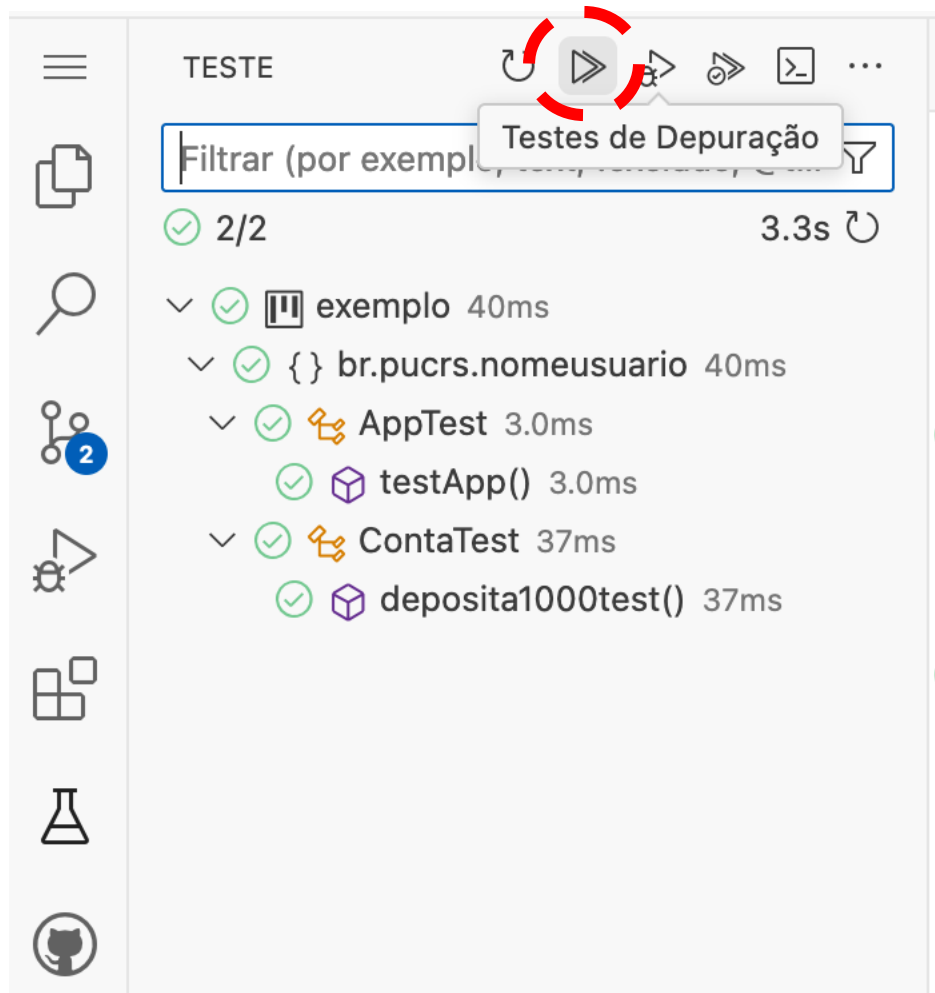
public class ContaTest {

    private Conta conta;

    @Test
    public void deposita1000test() {
        conta = new Conta(0.0);
        conta.depositar(1000.0);
        assertEquals(1000.0, conta.getSaldo());
    }
}

```

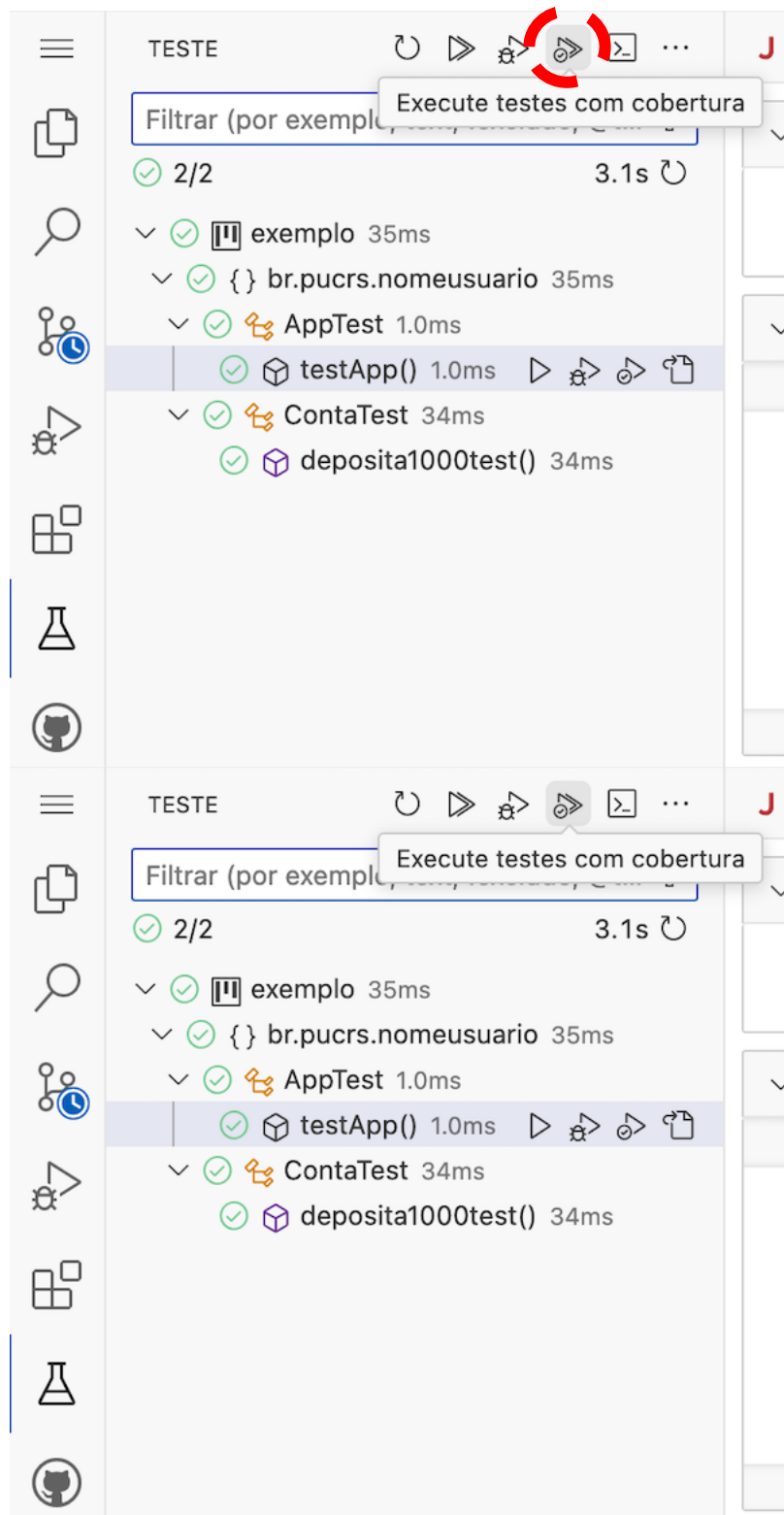
- q. Para executar testes, você pode selecionar o ícone de testes e depois clicar no botão de executar testes:



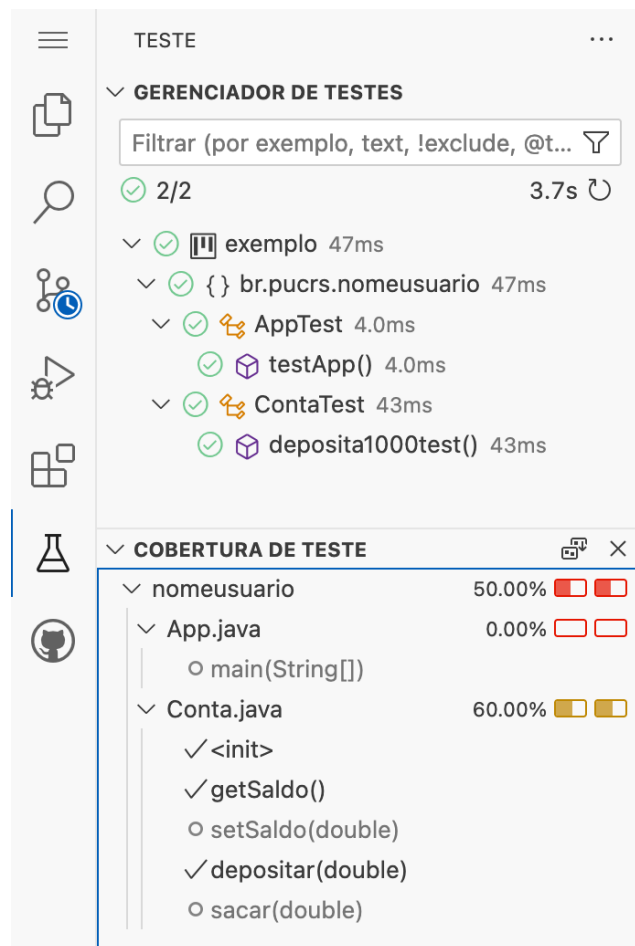
- r. Após a execução é possível verificar o resultado dos testes na própria janela de testes ou no relatório da execução – o ideal é que todos os testes passes (ícone verde).



- s. Uma outra facilidade que o JUnit permite é a análise de teste de cobertura – verificar se todo os métodos implementados foram testados.



t. Execute o teste de cobertura e veja o relatório.



- u. No relatório você pode clicar sobre uma classe para ver os métodos não testados. Clique na classe Conta.java do relatório e veja quais métodos não têm teste.

• Atividade – incrementar a classe Conta e implementar métodos de teste adicionais:

1. Na classe Conta implemente um método construtor sem parâmetros, neste caso o saldo inicial deve ser 0.0.
2. Na classe Conta implemente um método chamado 'transferir' que tenha 2 parâmetros (Conta destino, double valor) que retira o valor do saldo da conta atual e deposita o valor na conta destino.
3. Na classe de testes ContaTest implemente métodos de teste para que a cobertura de testes da classe Conta atinja 100%.

O roteiro descrito é uma introdução para o uso de JUnit em um projeto Java dentro do Codespaces.

Lembretes:

- Como o Codespaces executa de forma virtual, lembre-se que continuamente fazer commit e push para o GitHub.
- Ao final deste roteiro, não se esqueça de parar o Codespace com o comando 'Stop Current Codespace'